

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_05_)

Use of Venus Simulator for Register Level Execution of RISC-V Control Instructions

Student Name: __Areeb Fatima__ Roll No: __24I-6543__ Section: __B__

Date performed: __16th September__, 2026

MARKS AWARDED: ____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

The actual instructions are those which have conditions to come true for the execution of instruction while the pseudo instructions are those which are unconditional jumps and always goes to the target without seeing the condition to come true.

Code:

From Table 01:

The beq, blt, ble, bgt, bge and bne are all actual instructions in RISC-V. For e.g., beq, it's a conditional branch and executes only if the two registers are equal.

From Table 02:

The j instruction is a pseudo instruction as it's the unconditional jump and always goes to the target. While the jal and jalr are actual instructions.

Output:

Branches

type	target	# conditional branch to program label target
beq	t0, t1, target	# branch to target if t0 = t1
blt	t0, t1, target	# branch to target if t0 < t1
ble	t0, t1, target	# branch to target if t0 <= t1
bgt	t0, t1, target	# branch to target if t0 > t1
bge	t0, t1, target	# branch to target if t0 >= t1
bne	t0, t1, target	# branch to target if t0 <> t1

Table 1

Jumps

j	target	# unconditional jump to target
jal	ra, target	# unconditional jump to PC+target and save PC in ra
jalr	zero, const(ra)	# unconditional jump to ra+const and save PC in zero

Table 2

Task 2:

Description:
In this task, we use the beq and j instructions to understand the difference between the two as we took two numbers by using addi and then check if then equal by using beq then add the numbers otherwise j will execute and the subtraction will happen.
Code:
<pre>.data .text addi s0, x0, 5 addi s1, x0, 5 .globl main beq s0, s1, true j end true: add s2, s0, s1 end: sub s2, s0, s1</pre>
Output:

Run Step Prev Reset Dump Trace				(x15)	
PC	Machine Code	Basic Code	Original Code	a6 (x16)	0
0x0	0x00500413	addi x8 x0 5	addi s0, x0, 5	a7 (x17)	0
0x4	0x00500493	addi x9 x0 5	addi s1, x0, 5	s2 (x18)	10
0x8	0x00940463	beq x8 x9 8	beq s0, s1, true	s3 (x19)	0
0xc	0x0080006F	jal x0 8	j end	s4 (x20)	0
0x10	0x00940933	add x18 x8 x9	add s2, s0, s1	s5	0

Task 3:

Description:

In this task, we were given the two codes, and we must analyze them as in both we took the two words and checked if they equal then go to Fin, otherwise true will execute after that and then jump to end. In code 1, the words are not equal so true is executed and jump to end and in code 2, the words are equal, so we go to Fin and subtract those numbers

Code:

Code 01:

addi s0, s0, 4 # s0= s0+4

addi s1, s1, 6 # s1= s1+6

beq s0, s1, Fin # if s0=s1 then go to Fin

True: # always executes as 1

add s2, s1, s0 # s2= s1+s0

j end #always executes

Fin:

sub s2, s1, s0 # s2= s1-s0

end:

Code 02:

addi s0, s0, 4 # s0= s0+4

addi s1, s1, 4 # s1= s1+4

beq s0, s1, Fin # if s0=s1 then go to Fin

True: # always executes as 1

add s2, s1, s0 # s2= s1+s0

j end #always executes

Fin:

sub s2, s1, s0 # s2= s1-s0

end:

Output:

Code 01:

Run Step Prev Reset Dump Trace				
PC	Machine Code	Basic Code	Original Code	
0x0	0x00440413	addi x8 x8 4	addi s0, s0, 4 # s0= s0+4	a7 (x17) 0
0x4	0x00648493	addi x9 x9 6	addi s1, s1, 6 # s1= s1+6	s2 (x18) 10
0x8	0x00940663	beq x8 x9 12	beq s0, s1, Fin # if s0=s1 then go to Fin	s3 (x19) 0
0xc	0x00848933	add x18 x9 x8	add s2, s1, s0 # s2= s1+s0	s4 (x20) 0
0x10	0x0080006F	jal x0 8	j end #always executes	s5 (x21) 0
0x14	0x40848933	sub x18 x9 x8	sub s2, s1, s0 # s2= s1-s0	

Code 02:

Run Step Prev Reset Dump Trace				a7 (x17) 0
PC	Machine Code	Basic Code	Original Code	s2 (x18) 0
0x0	0x00440413	addi x8 x8 4	addi s0, s0, 4 # s0= s0+4	s3 (x19) 0
0x4	0x00448493	addi x9 x9 4	addi s1, s1, 4 # s1= s1+4	s4 (x20) 0
0x8	0x00940663	beq x8 x9 12	beq s0, s1, Fin # if s0=s1 then go to Fin	s5 (x21) 0
0xc	0x00848933	add x18 x9 x8	add s2, s1, s0 # s2= s1+s0	
0x10	0x0080006F	jal x0 8	j end #always executes	
0x14	0x40848933	sub x18 x9 x8	sub s2, s1, s0 # s2= s1-s0	

Task 4:

Description:

In this task, we were given a c program in which we initialize the 4 words in data and load address of the four words and then load the word and then we check for the condition that if a is equal to b then subtract d and e and else add e and d, we perform this by using the beq, sub and add operations and in last to end the loop we put a jump to end for stopping the execution.

Code:

```
.data

a: .word 5

b: .word 10

d: .word 4

e: .word 3

.text

.globl main

main:

la a0, a

lw t0, 0(a0)
```

```
la a1, b

lw t1, 0(a1)

la a2, d

lw t2, 0(a2)

la a3, e

lw t3, 0(a3)

beq t0, t1, Loop

else:

add s0, t2, t3

j end

Loop:

sub s0, t2, t3

end:
```

Output:

Run Step Prev Reset Dump Trace					
0x18	0x10000617	auipc x12 65536	la a2, d	tp (x4)	0
0x1c	0xFF060613	addi x12 x12 -16	la a2, d	t0 (x5)	5
0x20	0x00062383	lw x7 0(x12)	lw t2, 0(a2)	t1 (x6)	10
0x24	0x10000697	auipc x13 65536	la a3, e	t2 (x7)	4
0x28	0xFE868693	addi x13 x13 -24	la a3, e	s0 (x8)	7
0x2c	0x0006AE03	lw x28 0(x13)	lw t3, 0(a3)	s1 (x9)	0
0x30	0x00628663	beq x5 x6 12	beq t0, t1, Loop		
0x34	0x01C38433	add x8 x7 x28	add s0, t2, t3		
0x38	0x0080006F	jal x0 8	j end		

Task 5:

Description:

In this task, we were asked to shift a word to the left so for that we took a word in data then load its address and load the word after that and then we use the command of slli and then shift the word by 2 as if 3 is the word then by shifting its, 3×2^2 as 3 is multiplied with 2 in first shift and by shifting of two its again multiplied with two and then we exit from the loop in this way.

Code:

```
.data
a: .word 19

.text

.globl main

main:

la a0, a

lw t0, 0(a0)

slli s0, t0, 1

slli s1, s0, 1
```

Output:

Assemble & Simulate from Editor				tp (x4)
PC	Machine Code	Basic Code	Original Code	t0 (x5)
0x0	0x10000517	auipc x10 65536	la a0, a	19
0x4	0x00050513	addi x10 x10 0	la a0, a	t1 (x6)
0x8	0x00052283	lw x5 0(x10)	lw t0, 0(a0)	t2 (x7)
0xc	0x00129413	slli x8 x5 1	slli s0, t0, 1	s0 (x8)
0x10	0x00141493	slli x9 x8 1	slli s1, s0, 1	s1 (x9)

Task 6:

Description:
In this task, we were given a loop and we have to initialize counter to 0 so we took a word with 0 value and for i is equal to 1 to 5, for that we took another two words with value 0 and 5 and then load the values and run the loop and check for if ble, as if i is less than and equal to 5 then increment counter and i and check for the loop and if i is greater than 5 then exit.
Code:
<pre>.data counter: .word 0 i: .word 1 z: .word 5 .text .globl main main: la a0, counter lw t0, 0(a0) la a1, i lw t1, 0(a1) la a2, z lw t2, 0(a2) Loop: addi t0, t0, 1 addi t1, t1, 1 ble t1, t2, Loop</pre>
Output:

0xc	0x10000597	auipc x11 65536	la a1, i	t0 (x4)
0x10	0xFF858593	addi x11 x11 -8	la a1, i	t0 (x5)
0x14	0x0005A303	lw x6 0(x11)	lw t1, 0(a1)	t1 (x6)
0x18	0x10000617	auipc x12 65536	la a2, z	t1 (x6)
0x1c	0xFF060613	addi x12 x12 -16	la a2, z	t2 (x7)
0x20	0x00062383	lw x7 0(x12)	lw t2, 0(a2)	t2 (x7)
0x24	0x00128293	addi x5 x5 1	addi t0, t0, 1 #counter++	s0 (x8)
0x28	0x00130313	addi x6 x6 1	addi t1, t1, 1 #i++	s0 (x8)
0x2c	0xFE63DCE3	bge x7 x6 -8	ble t1, t2, Loop	s1 (x9)

Task 7:

Description:

In this task, we were given a c program in which we must declare an array with 10 values and then run the loop for if i is less than 5 then add 5 in i, for this we took the array by using .space 40 (space for 10 values) and then we initialize i to 0 by addi operation and then we add 5 to i in loop and then offset the array and load address of base array and add offset to array and then store and increment i and check by using blt as if i less than 5 then jump to the loop meanwhile when i=5 then exit.

Code:

```
.data
    num: .space 40 # reserve space for 10 integers
    a: .word 5

.text
    addi t0, x0, 0 # i=0

.globl main

    la a0, a
    lw t1, 0(a0) # t1=5

Loop:
```

```
addi s0, t0, 5 # i+5

slli s1, t0, 2 # offset

la s2, num # num[0]

add s3, s2, s1 # num+offset

sw s4, 0(s3) # num[i]

addi t0, t0, 1 # i++

blt t0, t1, Loop
```

Output:

0xc	0x00052303	lw x6 0(x10)	lw t1, 0(a0) # t1=5	t0 (x5)	5
0x10	0x00528413	addi x8 x5 5	addi s0, t0, 5 # i+5	t1 (x6)	5
0x14	0x00229493	slli x9 x5 2	slli s1, t0, 2 # offset	t2 (x7)	0
0x18	0x10000917	auipc x18 65536	la s2, num # num[0]	s0 (x8)	9
0x1c	0xFE890913	addi x18 x18 -24	la s2, num # num[0]	s1 (x9)	16
0x20	0x009909B3	add x19 x18 x9	add s3, s2, s1 # num+offset		
0x24	0x0149A023	sw x20 0(x19)	sw s4, 0(s3) # num[i]		
0x28	0x00128293	addi x5 x5 1	addi t0, t0, 1 # i++		
0x2c	0xFE62C2E3	blt x5 x6 -28	blt t0, t1, Loop		

Task 8:

Description:

In this task, we were given a nested loop in C and we have to write in assembly, so we first initialize the counter to 0 and x to 0 for outer loop then in outer loop we initialize y to 0 and in going to inner loop we add x and y in counter and then increment y by 1 and continue the loop for y is less than 2 and then after the inner loop ends we increment x by 1 and check for outer loop for x is less than 3. So, when x=3 then the outer loop ends and exit.

Code:

```
.data

a: .word 2
```

```
b: .word 3

.text

    addi s1, x0, 0 # counter=0

    addi s0, x0, 0 # x=0
```

```
.globl main

    la a0, a

    lw t0, 0(a0) # t0=2

    la a1, b

    lw t1, 0(a1) # t1=3
```

OuterLoop:

```
    addi s2, x0, 0 # y=0
```

InnerLoop:

```
    add s1, s0, s2 # counter=x+y

    addi s2, s2, 1 # y++

    blt s2, t0, InnerLoop

    addi s0, s0, 1 # x++

    blt s0, t1, OuterLoop
```

Output:

Run Step Prev Reset Dump Trace				(x4)
0x14	0x10000597	auipc x11 65536	la a1, b	t0 (x5) 2
0x18	0xFF058593	addi x11 x11 -16	la a1, b	t1 (x6) 3
0x1c	0x0005A303	lw x6 0(x11)	lw t1, 0(a1) # t1=3	t2 (x7) 0
0x20	0x00000913	addi x18 x0 0	addi s2, x0, 0 # y=0	s0 (x8) 3
0x24	0x012404B3	add x9 x8 x18	add s1, s0, s2 # counter=x+y	s1 (x9) 3
0x28	0x00190913	addi x18 x18 1	addi s2, s2, 1 # y++	
0x2c	0xFE594CE3	blt x18 x5 -8	blt s2, t0, InnerLoop	
0x30	0x00140413	addi x8 x8 1	addi s0, s0, 1 # x++	
0x34	0xFE6446E3	blt x8 x6 -20	blt s0, t1, OuterLoop	